Parallel Programming Languages
Fortran 2008 Programming
Exercise Notes

Fiona Reid, Alan Simpson

Introduction

This document contains Fortran 2008 exercises for EPCC's MSc in HPC or Data Science students. These are intended to provide an introduction to programming in Fortran 2008.

Instructions are given for compiling codes using the CPLab machines and Cirrus. We recommend that you use one of these machines.

Exercise 1: Hello World

A simple exercise to check that you can compile and run a Fortran 2008 program. Using a text editor of your choice (emacs, nedit, vi, etc) write a simple program to print "Hello World" on the screen. Check that you can compile your program using one of:

```
gfortran hello.f90 -o hello   # Use GNU Fortran Compiler, available on CPLab and Cirrus
ifort    hello.f90 -o hello   # Use Intel Fortran Compiler, only available on Cirrus
pgf90    hello.f90 -o hello   # Use PGI Fortran Compiler, only available on CPLab
```

The CPlab machines have both the GNU and PGI compilers loaded by default. Cirrus has the GNU compiler loaded by default. If you want to use the Intel compiler on Cirrus then before compiling you need to execute the following command to ensure that `ifort` is added to your path:

```
module load intel-compilers-16/16.0.3.210
```

The `-o` flag is used to specify the name of the executable in this case `hello`. The above statement will compile your code with the default level of optimisation which may not be very fast. If you want to compile an optimised version of your code adding `-O3` for the GNU or Intel compiler or `-fast` for the PGI compiler to your compile line usually gives good performance or timing results. E.g.

```
gfortran -O3   hello.f90 -o hello_opt
ifort    -O3   hello.f90 -o hello_opt
pgf90    -fast hello.f90 -o hello_opt
```

will produce an optimised version of the code with the name `hello_opt`. For a very simple code such as Hello World the performance will be very similar but for more complex codes you may see significant improvements when optimisation is used.

After successfully compiling your program execute it with `./hello` and check that "Hello World" appears on the screen.

Note: that emacs, nedit and vi all have "Fortran" modes. The Fortran modes provide features such as auto-indentation, syntax highlighting, parenthesis matching etc.

Exercise 2: Degrees to Radians

The Fortran 2008 intrinsic trigonometry functions (e.g. `sin()`, `cos()`, `tan()` etc) require their input to be in radians. In scientific applications results will often be provided in degrees. Therefore, it is useful to have a function which converts from degrees to radians and vice versa.

Write a module (modules will be explained in lecture 2) called `trig` which contains two functions: `degtorad` and `radtodeg`. `degtorad` should be a function which converts angles from degrees to radians, `radtodeg` should perform the inverse operation. A template for module `trig.f90` is provided below:

```
module trig
  implicit none
contains
  real function degtorad
    . . .
  end function degtorad

  real function radtodeg
    . . .
  end function radtodeg
end module trig
```

Write a program to read in 3 angles (in degrees) from the screen. Your program should *use* module `trig` to convert the 3 angles from degrees to radians and then back to degrees again. A template for program `angleconv.f90` is provided below:

```
program angleconv
  use trig
  implicit none

! read in three angles
! convert angles from degrees to radians
! print out the three angles
! convert angles radians to degrees
! print out the three angles

end program angleconv
```

Compile your program with `gfortran trig.f90 angleconv.f90 -o angleconv`

Hint: The value of π can be obtained from `4.0 * atan(1.0)`

Exercise 3: Square Roots

Write a program to calculate both roots of the quadratic equation:

$$ax^2 + bx + c = 0 \tag{1}$$

Your program should read the values of a, b and c from the command line and output both roots of equation 1 to the screen. You should include a conditional statement to check whether the discriminant, $b^2 - 4ac < 0$ and report an error message if this is the case.

Hint: recall that the roots of the quadratic equation are given by:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \tag{2}$$

Exercise 3a: Complex square roots

Modify your square root program to allow complex square roots to be computed. Your program should use the `complex` variable type. Verify that $a = 4$, $b = 1$, $c = 1$ gives roots:

$$x = -0.125 + 0.484i$$
$$x = -0.125 - 0.484i$$

Hint: you may wish to use the intrinsic procedures `cmplx(x)` or `aimag(c)`. `cmplx(x)` converts the real type variable `x` to a complex type variable. `aimag(c)` returns the imaginary part of the complex variable `c`.

Exercise 3b: Complex Square Roots - using subroutines and modules

Modify your square root program from exercise 3 so that the calculation of square roots is carried out inside a subroutine called `getroots`. You should think about the `intent` of the different variables e.g. which variables are input only, output only or both input and output?

Now encapsulate the subroutine `getroots` within a module `mod_roots.f90`.

After each modification verify that you still obtain the same answers for $a = 4$, $b = 1$ and $c = 1$.

Exercise 4: Discontinuous Function

Write a Fortran 2008 code to evaluate the discontinuous function:

$$\begin{aligned} 1 \le x < 400: &\quad y = x * x * x + C1 \\ 400 \le x < 1000: &\quad y = x * x + C2 \\ 1000 \le x \le 10000: &\quad y = x \end{aligned}$$

Declarations: declare two vector integer arrays, x and y, each with 10000 elements, plus two integer scalars, C1 and C2.

Initialisation: initialise the vector x to be

$$x(i) = i, \text{ for all values of i from 1 to 10000}$$

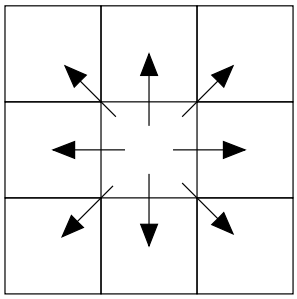
(ideally using an array constructor). Assign scalar constants C1=5 and C2=10.

Calculation: assign the corresponding elements of y. In Fortran 90, you can use either the array subscript notation to assign array sections or use the WHERE statements on the whole array.

Verification: Print out your answers for y(50), y(500) and y(1500). Check that your code gives the correct results (125005, 250010 and 1500 respectively).

Exercise 5: Game of Life

John Conway's "Game of Life" is a simple cellular automata. It lives on a 2D grid with periodic boundary conditions and cells have two states: dead or alive. The evolution of each cell is determined by simple rules involving its 8 neighbours:



- if a cell has exactly two alive neighbours it maintains state.
- if it has exactly three alive neighbours it is (or becomes) alive.
- otherwise, it is dead (or dies).

Write a Fortran program using array syntax to simulate the Game of Life. Your code will need to:

1. Initialise board

   Start loop

2. Print board
3. Calculate number of neighbours
4. if (neighbours = 3) then live (or become alive)
   if (neighbours $<$ 2) or (neighbours $>$ 3) then die (or remain dead)

   End loop

The number of neighbours can be calculated using shifts, e.g.

```
target = CSHIFT(source, SHIFT=shift, DIM=dimension)
```

sets target to be the same as source but with its elements shifted a distance shift along dimension dimension of the array source. For example,

```
target = CSHIFT(source, SHIFT=-1, DIM=1)
```

would set

```
target(i) = source(i - 1)
```

`CSHIFT` automatically performs periodic boundary conditions. Otherwise references would be made to elements outside the bounds of the array.

The Exercise

A template for this exercise (`life.f90`) is provided with the Parallel Programming Languages course package, which includes pointers to the completion of this exercise and also all the print statements necessary for viewing the results. The tar file containing the template code can be downloaded from your Learn entry for Parallel Programming Languages under the "Problem Sheets" `Fortran2008` folder. The file, `templates.tar`, contains the exercise template.

To unpack the tarball use:

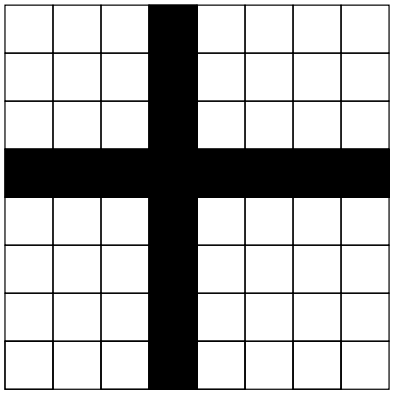
```
tar xvf templates.tar
```

This will create a directory called `Fortran2008_templates/` containing 1 file, e.g.

```
freid3$ tar xvf templates.tar
x Fortran2008_templates/
x Fortran2008_templates/life.f90
```

The `life.f90` file is a code template required for the Game of Life Exercise.

Initialisation: use array syntax to initialise a board (an $N \times N$ `INTEGER` array with value 1 for a live cell and 0 for a dead one) with the following pattern, where shaded cells are alive (set N = 8 to start). Set "alive" the row and column nearest the centre (i.e. N/2)



Update: declare an array the same size as the board ($N \times N$) to contain the number of neighbours for each point. Include all nearest neighbours, including diagonal neighbours. This update can be done using the following Fortran 90 features.

- Use `CSHIFT` to calculate the number of neighbours. In order to access the diagonal neighbours you may need to use nested `CSHIFT`s. For example,

  ```
  target=CSHIFT(CSHIFT(source,SHIFT=1,DIM=2),SHIFT=-1,DIM=1)
  ```

  This would shift array source initially in the 2nd dimension and then in the 1st.

- Use `WHERE` to decide whether to create or kill new organisms at each grid point.

Compilation: compile your code (board size N=8 for the test, for 10 iterations of evolution) and execute this on a single processor. Check the programme is working correctly by printing out the number of live elements at each iteration. Compare your results with those below.

Iteration	Live Cells
0	15
1	30
2	16
3	24
4	16
5	16
6	16
7	16
8	12
9	16
10	28

Visualisation: simple visualisation of the evolution of the system can be made by printing the board as either a plain text file (using the standard Fortran print format statements), or using the piece of code included in the template to produce a series of bitmaps which can be animated using widely available tools, such as display. Once run, the program with the suitable printing statements should produce a set of files, `life_*.pgm`, which can be viewed using display:
To use display to view the images use:

```
display -sample 200 -delay 1 *.pgm
```

where `-sample 200` increases the image size to 200 x 200 pixels and `-delay 1` animates the images with a 1 second delay. If the animation doesn't work then you can press the space bar to advance to the next image or click on the image with the right mouse button and select next.

Extra Exercise A

`CSHIFT` was used to count the number of neighbouring live cells. You can optimise the code to do the count with only four `CSHIFT` operations instead of eight / twelve. This requires the use of a temporary array the same size as board. For each array element,

- create the temporary by adding the current value of board to the value of the left and right partners (using two `CSHIFT`s);
- find the upper and lower partners for this temporary array (using two more `CSHIFT`s) and add these to the temporary array;
- each element now contains the number of neighbours, plus the original value of the cell. Subtract the original state to obtain the number of neighbours with four `CSHIFT`s.

Extra Exercise B

Try to count and display the number of alive/dead/new/killed cells at each iteration. You will probably need to use some additional Fortran 90 features.

Extra Exercise C

Define an extra array which is used for display. In this array keep track of the age of the cell. This array can be used for a grey scale display and so the age of the cell can be seen. To print out this grey scale array, change the header statement from

```
WRITE(10,fmt='(''P2'',/,I3,2X,I3,/,I3)') N, N, 1
```

to

```
WRITE(10,fmt='(''P2'',/,I3,2X,I3,/,I3)') N, N, loop
```

and the values printed will be displayed as grey scales (0-255).